Midterm Project: Cybersecurity Plan for an AI-Integrated IIoT System

IIoT Integrated System: Glucometer

Contributors:
Cassy Cormier
Monica Joya
Sufyan Rafiq
Kolapo Mogaji

Professor Sitaram Ayyagari
17181 ITAI 3377
4 April 2026

1. System Design and Vulnerability Identification

1.1 Introduction

This report presents a cybersecurity plan for a hypothetical AI-integrated IIoT glucometer system designed for educational analysis. The goal is to identify potential vulnerabilities across the system architecture and develop a comprehensive defense strategy aligned with industry standards including HIPAA, FDA Cybersecurity Guidance, and IEC 62443.

1.2 System Selection

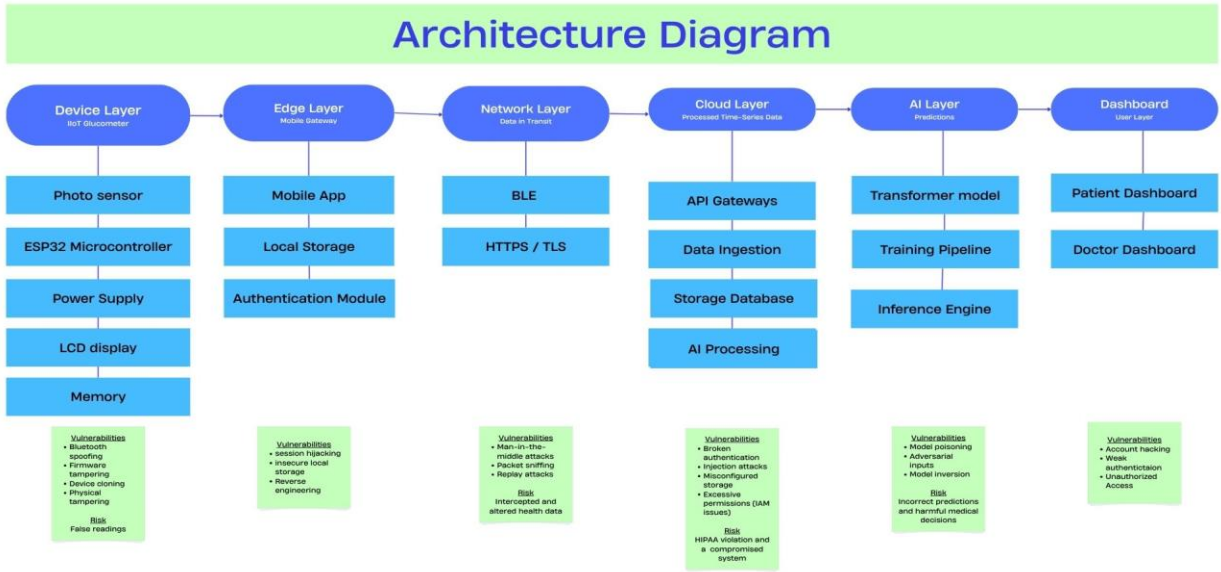
The IIoT-based glucometer consists of multiple connected layers:

- Device Layer: Photo sensor, power supply, ESP32 microcontroller, LCD display, memory
- Edge Layer: Mobile application, local storage, authentication module
- AI Layer: LSTM/Transformer model for glucose prediction
- Network Layer: Bluetooth (BLE), HTTPS/TLS
- Cloud Layer: APIs, data ingestion, storage, AI processing
- Dashboard Layer: Patient and clinician frontend interfaces

Data flows from the glucose device via BLE to the mobile app. Encrypted data is forwarded to the cloud infrastructure where the AI model analyzes it to form predictions, which are then sent to the dashboard UI.

1.3 Architecture Diagram

The architecture diagram (see Appendix) illustrates the six-layer structure with vulnerabilities identified at each layer: device-level attacks (Bluetooth spoofing, firmware tampering), edge-layer risks (session hijacking, insecure storage), network threats (MITM attacks, packet sniffing), cloud vulnerabilities (injection attacks, misconfigured storage), AI-specific attacks (model poisoning, adversarial inputs), and dashboard risks (broken authentication, unauthorized access).



2. Vulnerability Assessment

The following assessment identifies key vulnerabilities across six domains with their exploitation methods and risk levels.

2.1 Device Vulnerabilities

Vulnerability	Description & Exploitation	Risk	Impact
Firmware Tampering	ESP32 flash memory lacks protection; attacker uses JTAG/UART to reflash malicious firmware	Critical	False readings
Debug Interface Exposure	JTAG/UART left enabled; attacker extracts BLE keys and injects data	Critical	Data theft
Photosensor Spoofing	No tamper detection on analog signal; attacker injects light to fake readings	Critical	Wrong alerts

2.2 Network Vulnerabilities

Vulnerability	Description & Exploitation	Risk	Impact
BLE MITM Attack	'Just Works' pairing has no MITM protection; attacker sniffs/modifies packets with Ubertooth	Critical	Data interception
Insecure API Endpoints	Hardcoded API keys in firmware; attacker posts forged glucose readings directly	Critical	Data injection
Cloud Misconfiguration	S3 buckets publicly accessible; attacker downloads patient records via S3Scanner	Critical	PHI breach

2.3 Data Vulnerabilities

Vulnerability	Description & Exploitation	Risk	Impact
Insufficient Anonymization	Quasi-identifiers enable re-identification via linkage attacks	Critical	HIPAA violation
No Data Integrity Check	No HMAC on records; attacker modifies historical data to corrupt AI training	Critical	Model corruption

2.4 Application Vulnerabilities

Vulnerability	Description & Exploitation	Risk	Impact

Insecure Token Storage	JWT stored unencrypted in SharedPreferences; attacker extracts and replays token	Critical	Account takeover
API Injection	No input validation; NoSQL injection returns all patient records	Critical	Data exfiltration

2.5 AI Model Vulnerabilities

Vulnerability	Description & Exploitation	Risk	Impact
Adversarial Inputs	Crafted glucose sequences fool LSTM into predicting safe values during hypoglycemia	Critical	Missed alerts
Model Poisoning	Attacker injects malicious training samples creating persistent backdoor	Critical	Corrupted model

2.6 Human Factor Vulnerabilities

Vulnerability	Description & Exploitation	Risk	Impact
Patient Phishing	Spoofed SMS directs patient to fake login portal; credentials captured	High	Account compromise
Improper Disposal	Discarded device retains BLE keys and glucose history in flash memory	High	PHI exposure

3. Defense Strategy

The defense strategy addresses identified vulnerabilities across eight security domains, aligned with HIPAA, FDA Cybersecurity Guidance, and IEC 62443 standards.

3.1 Secure by Design

- Defense in Depth: Multiple security layers across hardware, firmware, transport, and cloud
- Least Privilege: Each component operates with minimum required permissions
- Zero Trust: Every request verified regardless of network location; no implicit trust

3.2 Authentication and Access Control

- Multi-Factor Authentication: Required for all users (patients, providers, admins)
- Role-Based Access Control: Patient (own data), Provider (assigned patients), Admin (system config)

- Short-Lived Tokens: 15-minute JWT access tokens with secure refresh rotation

3.3 Encryption and Data Protection

- AES-256 at Rest: All PHI encrypted with HSM-backed key management
- TLS 1.3 in Transit: Enforced on all communications; legacy protocols disabled
- Data Minimization: Glucose values pseudonymized at ingestion

3.4 Network Security

- Network Segmentation: Five zones (Device, Gateway, DMZ, Application, Data)
- BLE Hardening: LE Secure Connections with Numeric Comparison pairing
- API Protection: WAF with OWASP rules, rate limiting, schema validation

3.5 Secure Software Development

- SAST/DAST: Integrated into CI/CD pipeline (Semgrep, OWASP ZAP)
- SBOM: Software Bill of Materials tracked against CVE feeds
- Secure OTA: Firmware signed with ECDSA-256, served over TLS 1.3

3.6 Physical Security

- Tamper-Evident Enclosure: Physical tamper switch triggers alert
- Secure Boot: ESP32 Secure Boot V2 with RSA-4096 signature verification
- Secure Decommissioning: Cryptographic erase on device retirement

3.7 Security Monitoring

- SIEM: Centralized logging with Splunk/Sentinel
- Intrusion Detection: AWS GuardDuty + network IDS
- Incident Response: HIPAA-compliant breach notification workflow

3.8 AI Model Protection

- Adversarial Testing: Model tested against FGSM, PGD attacks
- Training Data Integrity: SHA-256 hashes with provenance tracking
- Inference Security: Rate limiting, output bounding (20-600 mg/dL)

4. Implementation Plan

The defense measures will be implemented across four phases over 16 weeks.

Phase	Focus	Key Deliverables
1	Device Security	Secure boot, flash encryption, BLE hardening
2	Network & Cloud	Segmentation, TLS 1.3, WAF, CSPM
3	Application & AI	SAST/DAST, model signing, adversarial testing
4	Monitoring & Compliance	SIEM, incident response, HIPAA audit

4.1 Key Technologies

- Device: ESP-IDF SDK, mbedTLS, Hardware Security Module (HSM)
- Network: AWS WAF, Security Hub, Suricata IDS
- Application: Semgrep (SAST), OWASP ZAP (DAST), CycloneDX (SBOM)
- AI: IBM Adversarial Robustness Toolbox, MLflow, TensorFlow Privacy
- Monitoring: Splunk/Sentinel SIEM, PagerDuty, AWS CloudTrail

5. Penetration Testing Simulation

5.1 Testing Environment

- ESP32 Simulator for device-layer testing
- FastAPI Backend for API simulation
- SQL Database for data layer testing
- Kali Linux Virtual Machine for attack execution

5.2 Attack Simulations and Results

Penetration testing was conducted on the system I built using tools including Kali Linux, Burp Suite, Wireshark, Nmap, and curl requests.

Several attacks were successfully mitigated due to design choices in the system:

- SQL injection attempts were blocked through Pydantic validation and ORM-based queries
- Data tampering (e.g., extreme glucose values) was rejected by backend constraints
- Invalid or fake authentication tokens were denied
- API fuzzing did not cause crashes, showing strong input handling

However, testing also revealed critical vulnerabilities:

- Missing authentication on endpoints such as /devices, /alerts, and /predictions, allowing unauthorized access
- MITM vulnerability, where data was transmitted in plain text and could be intercepted
- Device spoofing, where I was able to inject false glucose readings into the system
- Open ports, increasing the system's attack surface

5.3 Findings and Improvements

The main weaknesses identified in the system were:

- Incomplete authentication enforcement across endpoints
- Lack of encryption for data in transit
- No device identity verification mechanism
- Missing data integrity protections

To improve the system, I would implement:

- Authentication and authorization on all API endpoints
- TLS encryption for all communications
- Device identity verification before accepting data
- Data integrity checks such as HMAC
- Closing unused ports to reduce exposure

These improvements would significantly strengthen the system against both network and device-level attacks.

6. Individual Reflections

6.1 Cassy Cormier

This project was especially valuable because I not only analyzed the system but also designed and tested it myself. That gave me a much deeper understanding of how vulnerabilities actually arise in real implementations. While developing the system, I focused on functionality first, but during penetration testing I saw how small oversights—like missing authentication on certain endpoints or not enforcing encryption—created serious security risks. Being able to successfully perform attacks like device spoofing and traffic interception made the impact very real.

The most important skill I developed was thinking from a developer and attacker perspective. Building the system helped me understand how design decisions affect security, and testing it showed me how those decisions can be exploited. Going forward, I want to focus more on secure-by-design development, especially in AI-integrated systems, where both data integrity and model reliability are critical.

6.2 Monica Joya

This project taught me more about cybersecurity than I expected, especially for medical devices. I originally thought the biggest risks would be in the cloud or mobile app, but I learned that the ESP32 device itself has many attack surfaces, from debug ports to the photosensor. What stuck with me most is how vulnerabilities chain together. A compromised BLE connection can inject bad data that flows to the AI model, which then gives wrong predictions to a patient. That is why defense in depth matters.

My role was putting everything together and building the implementation plan. Coordinating everyone's work helped me see how real security teams operate. The skill I developed most was thinking about security from a planning perspective, not just knowing what a vulnerability is, but figuring out how to actually fix it with timelines and resources. Going forward, I want to learn more about AI security, especially how to build models that are harder to fool with adversarial inputs.

6.3 Sufyan Rafiq

This vulnerability assessment shifted how I think about security as a layered problem touching every part of a system. What surprised me most was how dangerous human factors are. A clinician sharing a password or a patient clicking a phishing link can make every encryption protocol irrelevant. Writing about adversarial inputs suppressing a hypoglycemia alert made healthcare cybersecurity feel real. I was thinking about a patient who does not get warned in time. The skill I developed most was threat modeling, thinking like an attacker systematically. I want to explore

adversarial machine learning further, specifically building LSTM architectures robust to adversarial inputs in safety-critical applications.

6.4 Kolapo Mogaji

Designing defenses for a resource-constrained ESP32 while securing a HIPAA-regulated cloud backend was the biggest challenge. I could not deploy enterprise-grade controls on a device with limited flash and processing power, so I had to make trade-offs. The usability dimension was also challenging. A diabetic patient should not face security that feels like a bank vault. Every MFA step or short session TTL has a human cost for someone who may be experiencing a hypoglycemic episode. I learned about IEC 62443's Zone and Conduit models. If implementing this system, I would prioritize federated learning so the AI model can learn without centralizing PHI.

References

Cybersecurity and Infrastructure Security Agency. (2021). *Securing the healthcare and public health sector: Cybersecurity best practices*. https://www.cisa.gov/topics/cybersecurity-best-practices

MIT School of Engineering. (n.d.). *How do glucometers work?* https://engineering.mit.edu/engage/ask-an-engineer/how-do-glucometers-work/

National Institute of Standards and Technology. (2020). *Zero trust architecture* (Special Publication 800-207). https://csrc.nist.gov/publications/detail/sp/800-207/final

National Institute of Standards and Technology. (2023). *Guide to operational technology (OT) security* (Special Publication 800-82, Rev. 3). https://csrc.nist.gov/publications/detail/sp/800-82/rev-3/final

Open Web Application Security Project. (2018). *IoT security verification standard*. https://owasp.org/www-project-internet-of-things/

U.S. Department of Health and Human Services. (2013). *HIPAA security rule*, 45 C.F.R. Part 164. https://www.hhs.gov/hipaa/for-professionals/security/index.html

U.S. Food and Drug Administration. (2023). *Cybersecurity in medical devices: Quality system considerations and content of premarket submissions*. https://www.fda.gov/regulatory-information/search-fda-guidance-documents/cybersecurity-medical-devices-quality-system-considerations-and-content-premarket-submissions